dUltra: Ultra-Fast Diffusion Language Models via Reinforcement Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Masked diffusion language models (MDLMs) offer the potential for parallel token generation, but most open-source MDLMs decode fewer than 5 tokens per model forward pass even with sophisticated sampling strategies, limiting their parallel generation potential. Existing acceleration methods either rely on fixed confidence-based heuristics or use distillation-based approaches that fine-tune MDLMs on trajectories generated by a base model, which can become off-policy during fine-tuning and restrict performance to the quality of the base model's samples. We propose `dUltra`, an on-policy reinforcement learning framework based on Group Relative Policy Optimization (GRPO) that learns unmasking strategies for efficient parallel decoding. dUltra introduces an unmasking planner head that predicts per-token unmasking probabilities under independent Bernoulli distributions. We jointly optimize the base diffusion LLM and the unmasking order planner using reward signals combining verifiable reward, distillation reward, and the number of unmasking steps. Across mathematical reasoning and code generation tasks, dUltra achieves superior accuracy-efficiency trade-offs compared to the state-of-the-art heuristic baseline (Fast-dLLM) and distillation baselines (d3LLM, dParallel), demonstrating that learned unmasking trajectories through on-policy RL enable better exploitation of parallel generation in MDLMs. Code and checkpoints will be released upon acceptance.

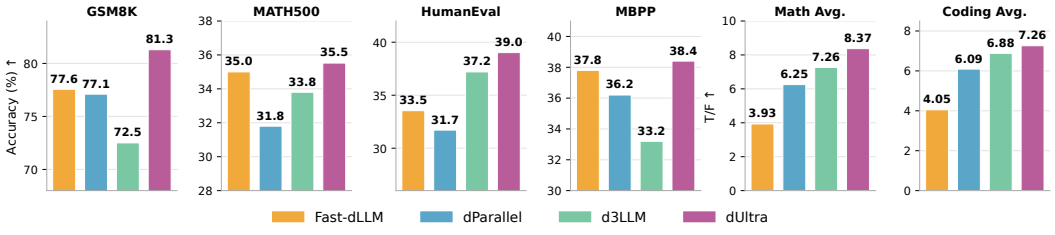

Figure 1: **dUltra accuracy-efficiency summary.** Across math reasoning and code generation benchmarks, dUltra consistently achieves strong task accuracy while decoding more tokens per model forward pass than Fast-dLLM, dParallel, and d3LLM. We report the main setting with generation length 256 and block size $B = 32$; complete $B = 32$ and $B = 128$ results are provided in Tables 1 and 3. T/F denotes generated tokens per model forward pass, so higher values indicate more parallel decoding.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

The great success of diffusion models and their parallel generation capabilities makes them attractive for language generation. There has been an ongoing effort to transfer the diffusion paradigm to the natural language domain. Earlier attempts tried to sample in the token embedding space [Li et al., 2022, Gulrajani and Hashimoto, 2023, Gong et al., 2023] or the latent space of a language autoencoder [Lovelace et al., 2023] using continuous diffusion. In a parallel line of work, discrete diffusion in the vocabulary space is studied in [Austin et al., 2021, Campbell et al., 2022, Meng et al., 2022, He et al., 2022, Wang and Cho, 2019]. Without the need for an invertible token-wise embedding function, discrete diffusion language models generally perform better than continuous diffusion language models when operating in a discrete state space with a countable number of states [Lou et al., 2023, Shi et al., 2024, Sahoo et al., 2024]. Moreover, it is observed that the best performance is achieved with an absorbing source distribution, which further restricts possible sampling paths [Lou et al., 2023, Austin et al., 2021]. The diffusion models with an absorbing source distribution are also called masked diffusion language models (MDLMs) because of the introduction of a masked token. Moreover, Ou et al. [2025] and Sahoo et al. [2024] have shown that the optimal solution that minimizes negative ELBO is independent of the time variable, thus promoting time-agnostic parameterization. Recent works have scaled time-agnostic MDLMs to show that their performance can rival that of autoregressive (AR) models with similar parameter size [Nie et al., 2025, Ye et al., 2025].

However, even the best open-source masked diffusion language models (MDLMs) [Nie et al., 2025, Ye et al., 2025] face the same slow sampling problem as continuous diffusion models. Apart from system-level optimization, such as approximate KV Cache [Wu et al., 2025, Hu et al., 2025], attempts to accelerate MDLMs have focused on designing confidence/entropy-aware sampling methods to unmask tokens in parallel [Ben-Hamu et al., 2025, Wu et al., 2025]. These deterministic parallel decoding strategies generally favor tokens with high confidence or low entropy within a single unmasking step, leading to 3-5x faster sampling speed. Despite this decent speed-up, the number of tokens decoded in parallel within a single denoising step is still low ($\sim$4.5 tokens per step on GSM8K and $\sim$3.3 tokens per step on MATH500) [Wu et al., 2025]. Meanwhile, commercial diffusion language models [Google DeepMind, 2025, Khanna et al., 2025] operate at a throughput of $\sim$1000 tokens/s, significantly surpassing the state-of-the-art throughput of autoregressive models, which typically range from 100-300 tokens/s [Khanna et al., 2025]. Therefore, current open-source MDLMs have yet to exploit the parallel generation potential of diffusion language models to their fullest and have not achieved "diffusion supremacy" (analogous to quantum supremacy) over autoregressive models.

A fundamental observation is that when tokens are generated in parallel, MDLMs assume conditional independence between them given the masked input sequence (see also Section 2.1) due to the curse of dimensionality [Aaron, 2024]. If the tokens are actually dependent and ambiguous[1], the actual distribution sampled from will have unwanted modes, and the model may generate nonsensical text. Since acceleration methods aim to increase parallelism by decoding more tokens per step, they inevitably unmask some dependent tokens together, violating the independence assumption. Therefore, given the current paradigm of parallel decoding[2], any method that accelerates sampling is implementing some form of mode filtering, i.e. the accelerated sampling procedure will try to sample from a single mode that produces sensible text out of all modes that a slower model can sample from. Appendix C illustrates this behavior.

Recent work [Qian et al., 2025, Chen et al., 2025] on accelerating diffusion LLM sampling with offline distillation demonstrates this principle as well. Specifically, d3LLM [Qian et al., 2025] distills from the sampling trajectory produced by the teacher model, effectively teaching the student model to sample from one of the high probability teacher modes at each sampling step. Similarly, dParallel [Chen et al., 2025] proposes certainty-forcing distillation that fine-tunes the model to be more confident on self-generated trajectories, which are fixed per prompt. This effectively encourages the model to sample from one of the high probability teacher modes marginalized by the prompts. However, this self-distillation approach faces two shortcomings. First of all, the quality of training data cannot exceed the capability of the base model. In other words, the training data only includes greedily sampled trajectories, limiting exploration and performance of the distilled model. Secondly,

[1] An ambiguous token means that multiple words in the vocabulary are predicted to have similar and high probability.

[2] Unless the curse of dimensionality is solved

the self-generated trajectory can be off-policy during distillation, and the student learns in contexts more frequently encountered by the teacher model rather than by itself. Therefore, we want a method that explores the teacher model modes and trains on samples generated by the student model. On-policy reinforcement learning naturally addresses both limitations: by training on trajectories generated by the current policy, it avoids the distribution mismatch inherent in offline distillation, while the exploration encouraged by RL allows discovering better unmasking strategies beyond those demonstrated by the base model.

Another implication of the observation above is that we can accelerate sampling while reducing the need for mode filtering by planning trajectories with more conditionally independent tokens per step. A one-step generation case study in Appendix D shows that LLaDA-Instruct decodes randomly masked tokens much more reliably than the same number of consecutive suffix masks, illustrating the value of conditional independence-aware planning. Our approach learns adaptive unmasking strategies throughout the full denoising trajectory to maximize the number of conditionally independent tokens decoded at each step.

Taking these motivations all together, in this work, we propose dUltra, a framework that *learns* adaptive unmasking strategies through on-policy RL combined with on-policy distillation reward. The headline accuracy-efficiency results are summarized in Figure 1, and an overview of the dUltra architecture is shown in Figure 2. Our key contributions are:

- We introduce an unmasking planner head that predicts per-token unmasking probabilities using independent Bernoulli distributions, enabling learnable sampling trajectory planning.
- We develop a GRPO-based on-policy distillation framework that jointly optimizes the diffusion model and unmasking order planner using both verifiable reward and distillation reward. We also propose advantage clipping to enable effective RL training of the unmasking planner head.
- We show state-of-the-art performance in terms of both tokens per model forward pass (T/F) and number of function evaluations (NFE) on mathematical reasoning (GSM8K, MATH500) and code generation (HumanEval, MBPP) tasks.

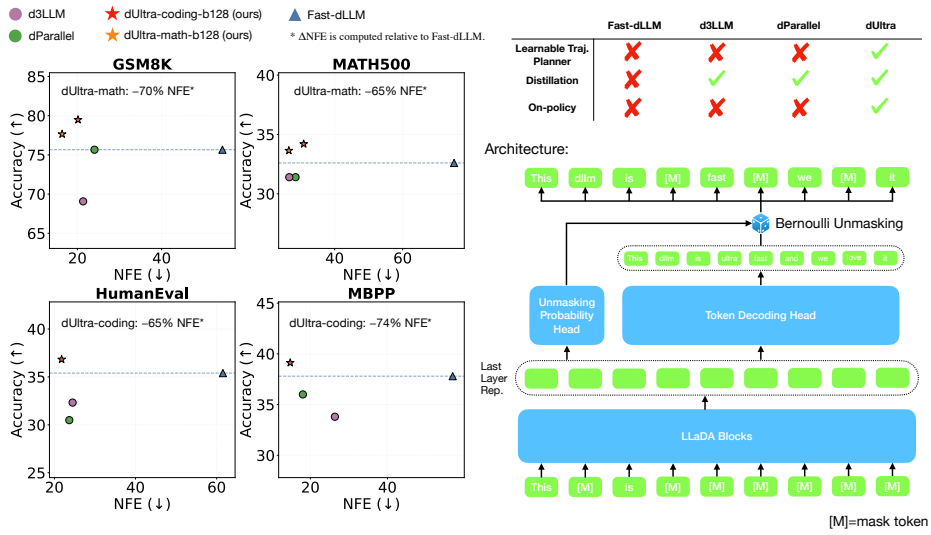

Figure 2: **dUltra overview.** Left: Accuracy versus number of function evaluations (NFE; denoising steps) on math and coding benchmarks for Fast-dLLM, dParallel, d3LLM, and dUltra (ours) variants. Here we use a block size of 128 and a generation length of 256. Right: dUltra architecture: a masked diffusion LM (LLaDA blocks) produces last-layer representations that feed (i) a token decoding head to output logits for masked positions and (ii) an unmasking planner head to output per-position unmasking probabilities; the Bernoulli sampled unmasking decisions update the masked sequence ([M] denotes the mask token).

2 Preliminaries

Continuous-time MDLMs [Shi et al., 2024, Sahoo et al., 2024] define a forward noising process $q_{t|s}$ that progressively masks tokens according to a noise schedule α_t (with $\alpha_0 = 1$, $\alpha_1 = 0$), and a learned reverse process $p_\theta(x_s|x_t)$ that predicts clean tokens from masked inputs. We use $\mathcal{X} = \{1, \dots, m\}$ as the token vocabulary with the mask token as the m-th entry, and $\mathrm{Cat}(\boldsymbol{\pi})$ to denote a categorical distribution over the simplex Δ^m. Full single-token forward/reverse process formulations are given in Appendix M.

2.1 Sequence Factorization

For a token sequence $\boldsymbol{x} \in \mathcal{X}^L$ of length L, MDLM factorizes the forward and reverse processes to sample each token independently. For the forward process, $q_{t|0}(\boldsymbol{x}_t|\boldsymbol{x}_0) = \prod_{l=1}^{L} q_{t|0}(x_t^{(l)}|x_0^{(l)})$. For the reverse process, $q_{s|t,0}(\boldsymbol{x}_s|\boldsymbol{x}_t, \boldsymbol{x}_0) = \prod_{l=1}^{L} q_{s|t,0}\left(x_s^{(l)}|x_t^{(l)}, x_0^{(l)} \approx \boldsymbol{\mu}_\theta^{(l)}(\boldsymbol{x}_t, t)\right)$. Note that here $\boldsymbol{\mu_\theta}$ is dependent on the entire sequence, so this factorization assumes conditional independence.

2.2 Training Objective

For MDLM with carry-over unmasking [Sahoo et al., 2024, Shi et al., 2024, Nie et al., 2025, Ye et al., 2025], the NELBO reduces to a weighted cross-entropy over masked positions:

$$\mathcal{L}_{\text{NELBO}} = -\int_0^1 \mathbb{E}_{q(x_t|x_0)} \left[\frac{\alpha_t'}{1-\alpha_t} \sum_{i=0}^{|\boldsymbol{x}|} \mathbf{1}[\boldsymbol{x}_t^{(i)} = m] \cdot \log \boldsymbol{\mu_\theta}(\boldsymbol{x}_t; \boldsymbol{x}_t^{(i)} = \boldsymbol{x}_0^{(i)}) \right] dt, \tag{1}$$

where $\boldsymbol{\mu_\theta}(\boldsymbol{x}_t; \boldsymbol{x}_t^{(i)} = \boldsymbol{x}_0^{(i)})$ is the model's predicted probability for the i-th token being its ground-truth value. The full NELBO decomposition is in Appendix M.

2.3 Connection to Any-Order Autoregressive Modeling

Any-order autoregressive models (AO-ARMs) [Uria et al., 2014, Hoogeboom et al., 2021, Shih et al., 2022] evaluate the likelihood of a sequence $\boldsymbol{x}$ by marginalizing over all possible generation orders. More specifically, the expected negative log-likelihood minimized by AO-ARMs during training is

$$\mathcal{L}_\theta(\boldsymbol{x}) = -\mathbb{E}_{\xi \sim U(S_{|\boldsymbol{x}|})} \left[\sum_{k=1}^{|\boldsymbol{x}|} \log p_\theta(\boldsymbol{x}^{\xi(k)} | \boldsymbol{x}^{\xi(<k)}) \right], \tag{2}$$

where $U(S_{|\boldsymbol{x}|})$ is the uniform distribution over all permutations of the initial mask tokens, and each permutation uniquely specifies a sampling order. Works have shown that this loss is equivalent to the NELBO loss of MDLMs with carry-over unmasking (Equation (1)) under mild conditions [Ou et al., 2025, Kim et al., 2025]. Therefore, MDLMs with carry-over unmasking can be interpreted as AO-ARMs that sample generation orders from a uniform distribution over all possible permutations.

3 Methods

3.1 Path Planning with Unmasking Probability

If we strictly follow the predetermined forward noising process and corresponding time reversal (Appendix M) during denoising, a masked token has a *fixed* probability of $\frac{1-\alpha_{s(i)}}{1-\alpha_{t(i)}}$ remaining a mask token from time $t(i) = i/T$ to time $s(i) = (i-1)/T$ given the noise schedule function α. However, there is no inherent restriction on what the noise schedule function should be or what variable it should depend on under the framework of diffusion models. While maximizing ELBO requires sampling from the forward kernel $q(x_t|x_0)$ during training, at inference time we have flexibility in designing the reverse transition kernels $q_{s|t}$, allowing us to explore different unmasking strategies beyond the training distribution. One possibility is to make the schedule state-dependent by conditioning $\frac{1-\alpha_{s(i)}}{1-\alpha_{t(i)}}$ on the current masked state $\boldsymbol{x}_t$, resulting in per-token schedules $\frac{1-\boldsymbol{\alpha}^l_{s(i)}(\boldsymbol{x}_t)}{1-\boldsymbol{\alpha}^l_{t(i)}(\boldsymbol{x}_t)}$ for the l-th token. In

this way, the reverse denoising process is essentially a Markovian process with the transition kernel $p_{\boldsymbol{\theta}}(x_{s(i)}|x_{t(i)})$.

From the perspective of any-order autoregressive modeling, this is equivalent to learning a better distribution over sampling orders instead of the uniform distribution used in the standard negative log-likelihood. As motivated in Figure 5, we would like to refine this distribution to concentrate the probability mass over sampling orders that maximize the average number of conditionally independent tokens to unmask per step. Therefore, we want to learn a better unmasking order planner that can decide which tokens to unmask at each denoising step.

We therefore introduce an unmasking order planner to decide which mask tokens to unmask and with what probability. To decide which tokens to unmask at each step, we model per-token unmasking decisions as independent Bernoulli distributions, mirroring the conditional independence assumption in MDLM's token prediction factorization (Section 2.1). Modeling the full joint distribution over all possible position subsets would be intractable due to the exponential number of possibilities (2^n subsets for n masked tokens). While we sample unmasking decisions independently for computational feasibility, the planner can still capture dependencies between positions through self-attention when computing the unmasking probabilities. We introduce an unmasking planner head ϕ that takes the last-layer hidden representations of the base MDLM and outputs per-token unmasking probabilities:

$$p_{\text{unmask}}(x) = \sigma(\phi(h_x, x)), \qquad z_x \sim \text{Bernoulli}(p_{\text{unmask}}(x)), \tag{3}$$

where h_x is the last-layer hidden representation of the sequence of tokens x, and $z_x \in \{0, 1\}$ denotes the binary decision of whether to unmask token x.

The unmasking planner head ϕ is a lightweight module consisting of a single transformer block (matching the base MDLM architecture), time embedding layers for timestep conditioning, adaptive layer normalization, and a final linear projection to scalar logits. This design keeps the computational overhead minimal while allowing the planner to capture dependencies between masked positions and incorporate temporal information when deciding which tokens to unmask. The planner processes all masked token representations in parallel and outputs conditionally independent unmasking probabilities for each token. See Section N.1.1 for complete architectural details.

3.2 Exact Likelihood Calculation

One of the advantages of introducing an explicit unmasking planner head is that we can tractably compute the likelihood of each rollout during training, which is essential for policy gradient methods. Recall from the AO-ARM perspective (Section 2.3) that MDLMs define a Markovian generative process: at each step, we unmask a subset of tokens conditioned on all previously unmasked tokens. This Markovian structure makes the likelihood calculation straightforward—we simply multiply the transition probabilities at each step. Next we compute the transition probability at each denoising step.

We denote the prompt as $\mathbf{q}$ and the sequence completion after denoising step k as $\mathbf{o}^k$, and $\mathbf{o}^0$ is the completely masked sequence. At step k, we select positions $\mathbf{Pos}^{(k)} \subseteq \mathcal{X}_{\text{mask}}^{(k)}$ to unmask from the set of currently masked tokens $\mathcal{X}_{\text{mask}}^{(k)}$. Our model consists of two key components: (1) π_{token}, the token probability distribution for masked positions, which we obtain from the logits of the model output, and (2) π_{select}, the unmasking order planner that unmasks each token with probability p_{unmask} at each step. With these components, we model the parallel decoding procedure as an ancestral sampling process. We first sample the positions to unmask from π_{select}, and then sample the tokens for the selected positions from π_{token}. Therefore, the likelihood of generating $\boldsymbol{o}^k$ from $\boldsymbol{o}^{k-1}$ given prompt $\boldsymbol{q}$ is:

$$p\big(\boldsymbol{o}^k \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}\big) = \pi_{\text{select}}\big(\mathbf{Pos}^{(k)} \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}\big) \cdot \pi_{\text{token}}\big(\boldsymbol{o}^k \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}, \mathbf{Pos}^{(k)}\big) \tag{3}$$

The token selection probability factorizes as:

$$\pi_{\text{token}}\big(\boldsymbol{o}^k \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}, \mathbf{Pos}^{(k)}\big) = \prod_{j=1}^{\#\text{unmask}} \pi_\theta\big(\boldsymbol{o}^k_{\mathbf{Pos}_j^{(k)}} \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}, \mathbf{Pos}^{(k)}\big)$$

The position selection probability follows the product of independent Bernoulli distributions:

$$\pi_{\text{select}}\big(\mathbf{Pos}^{(k)} \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}\big) = \prod_{x \in \mathbf{Pos}^{(k)}} p_{\text{unmask}}(x) \cdot \prod_{x \in \mathcal{X}_{\text{mask}}^{(k)} \setminus \mathbf{Pos}^{(k)}} \big(1 - p_{\text{unmask}}(x)\big) \tag{4}$$

In practice, one can additionally incorporate a block size hyperparameter B to restrict the set of candidate positions: instead of selecting from all masked tokens $\mathcal{X}_{\text{mask}}^{(k)}$, we constrain $\mathbf{Pos}^{(k)}$ to be a subset of the leftmost B masked tokens. The block size interpolates between completely learnable unmasking order and autoregressive-like order: larger B allows the planner to select from positions throughout the sequence, while smaller B restricts candidates to the leftmost masked tokens, enforcing a more left-to-right progression similar to autoregressive decoding, though the planner retains flexibility in ordering tokens within each block.

Hence, the overall likelihood of a complete rollout with K denoising steps is:

$$p(\boldsymbol{o}^K \mid \boldsymbol{q}) = \prod_{k=1}^{K} p(\boldsymbol{o}^k \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}) = \prod_{k=1}^{K} \pi_{\text{select}}(\mathbf{Pos}^{(k)} \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}) \cdot \pi_{\text{token}}(\boldsymbol{o}^k \mid \boldsymbol{q}, \boldsymbol{o}^{k-1}, \mathbf{Pos}^{(k)}). \tag{5}$$

Equivalently, we can view each token's unmasking trajectory as following a non-homogeneous geometric distribution[3]: at each denoising step, the planner assigns a state-dependent probability of unmasking, and the token remains masked until it is eventually sampled to be unmasked.

3.3 Initializing On-policy Planners with Heuristic Methods

Randomly initialized planners often unmask too many tokens at the start of training, producing low-quality rollouts before RL can explore useful trajectories. We therefore initialize the planner by supervised imitation of the confidence-based Fast-dLLM decoder [Wu et al., 2025], treating its selected positions as binary targets. This gives GRPO a strong heuristic policy as a starting point, after which on-policy optimization can improve beyond fixed confidence-based sampling. The full initialization objective is given in Appendix F.

3.4 On-policy Distillation Reward

As motivated by Figure 4, to further accelerate sampling with parallel decoding beyond sampling conditionally independent tokens, the model must learn to sample from a single high-quality mode among multiple plausible modes to avoid generating incoherent text, i.e. the model should exhibit mode-seeking behavior. To encourage this behavior, we introduce an on-policy distillation reward that encourages the student diffusion LLM to maximize the sequence-level log-likelihood of on-policy samples evaluated by an autoregressive teacher model.

We use domain-specific teacher models to provide high-quality supervision signals: Qwen2.5-Math-7B-Instruct [Yang et al., 2024] for mathematical reasoning and Qwen2.5-Coder-7B-Instruct [Hui et al., 2024] for code generation. Given a prompt $\boldsymbol{q}$ and a completion $\boldsymbol{c}$ generated by the student model during rollout, we compute the on-policy distillation reward as:

$$r_{\text{distill}}(\boldsymbol{q}, \boldsymbol{c}) = \frac{1}{|\boldsymbol{c}|} \left(\left[\sum_{i=1}^{|\boldsymbol{c}|} \log p_{\text{teacher}}(c_i \mid \boldsymbol{q}, \boldsymbol{c}_{<i}) \right] - \beta \log p_{\text{student}}(\boldsymbol{c} \mid \boldsymbol{q}) \right) \tag{6}$$

where $|\boldsymbol{c}|$ is the length of the completion in tokens, $p_{\text{student}}(\boldsymbol{c} \mid \boldsymbol{q})$ is given by Equation (5), and β controls the weight of the entropy regularization term. When $\beta = 1$, maximizing $\mathbb{E}_{\boldsymbol{c} \sim p_{\text{student}}}[r_{\text{distill}}]$ corresponds to minimizing a scaled reverse KL divergence $\frac{1}{|\boldsymbol{c}|} D_{\text{KL}}(p_{\text{student}} \| p_{\text{teacher}})$. Reverse KL divergence is known to encourage mode-seeking behavior [Minka et al., 2005]: when the student places probability mass where the teacher model has low probability, the term $\log(p_{\text{student}}/p_{\text{teacher}})$ becomes large and heavily penalizes the divergence. We set $\beta = 0$ in our experiments, simplifying the reward to average teacher log-likelihood, while still encouraging mode-seeking behavior: the student learns to generate completions that the teacher assigns high probability, focusing on high-quality modes verified by the teacher model rather than exploring low-probability regions that may lead to incoherent text. Empirically, we did not find significant differences in performance when varying β.

The on-policy distillation reward is combined with verifiable rewards and efficiency rewards (i.e., negative number of denoising steps) via a weighted sum:

$$r_{\text{total}} = \lambda_{\text{task}} \cdot r_{\text{task}} + \lambda_{\text{distill}} \cdot r_{\text{distill}} + \lambda_{\text{step}} \cdot r_{\text{step}} \tag{7}$$

[3] Unlike the standard geometric distribution with constant success probability, the non-homogeneous variant allows the probability to vary across steps.

where the weights $\lambda_{\text{task}}, \lambda_{\text{distill}}, \lambda_{\text{step}}$ control the relative importance of each component. This multi-objective reward encourages the model to generate correct, coherent, and efficiently-sampled outputs. See Section N.2 for detailed reward configuration for each task.

3.5 GRPO Training Framework

We train the diffusion LLM and unmasking planner head using Group Relative Policy Optimization (GRPO), a policy-gradient method that operates on groups of trajectories per prompt and uses group-relative advantages as baselines. We use on-policy updates (`num_iterations`$= 1$) as off-policy updates provide no computational benefit: unlike autoregressive LLMs that evaluate likelihoods in a single forward pass, each likelihood evaluation using Equation (3) requires a complete denoising process. For each group, we sample a single prompt $\boldsymbol{q}$ and generate G trajectories with the denoised sequence at step k of the g-th trajectory denoted as $\boldsymbol{o}^{g,k}$. The group-relative advantages of these trajectories are calculated as $A^g = r^g - \text{mean}(r^{1:G})$. Note that we do not normalize the advantage to avoid introducing any bias into the policy gradient estimation [Liu et al., 2025b]. Then the GRPO objective reduces to the REINFORCE-style loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{|\boldsymbol{o}| \cdot G} \sum_{g=1}^{G} A^g \sum_{k=1}^{K^g} \log p_\theta\big(\boldsymbol{o}^{g,k} \mid \boldsymbol{q}, \boldsymbol{o}^{g,k-1}\big), \tag{8}$$

where $p_\theta(\boldsymbol{o}^{g,k} \mid \boldsymbol{q}, \boldsymbol{o}^{g,k-1})$ is the rollout likelihood defined in Equation (3), and K^g is the total number of denoising steps for the g-th trajectory. In practice, we use the ratio formulation $\frac{p_\theta}{\text{sg}(p_\theta)}$ instead of $\log p_\theta$ for numerical stability, where $\text{sg}(\cdot)$ denotes stop-gradient. The two formulations have identical gradients.

Advantage clipping. Vanilla GRPO can collapse the planner into never unmasking any token, because low-advantage conservative trajectories uniformly penalize both bad and correct unmasking decisions. We therefore clip below-threshold advantages:

$$A^g \leftarrow \begin{cases} 0 & \text{if } A^g < C \\ A^g & \text{otherwise} \end{cases} \tag{9}$$

where C is the clipping threshold. This ignores below-threshold trajectories and lets the planner learn primarily from successful unmasking strategies; we set $C = 0$ in experiments. Additional intuition and stability evidence are provided in Appendix G and Appendix H.

In summary, the complete training algorithm can be found in Appendix O.

4 Results

We evaluate dUltra on mathematical reasoning and code generation benchmarks to demonstrate its ability to achieve competitive performance with significantly reduced computational cost. All of our experiments use LLaDA-8B-Instruct as the base model. We compare against *state-of-the-art* confidence-based heuristics, such as Fast-dLLM [Wu et al., 2025], and *state-of-the-art* off-policy distillation-based methods, such as d3LLM [Qian et al., 2025] and dParallel [Chen et al., 2025]. We report both task performance and the average number of function evaluations (NFE), i.e., the number of denoising steps during generation.

Unlike distillation baselines, which rely on self-generated or heuristic trajectories without an external teacher signal, dUltra uses on-policy rollouts with an AR teacher model (Qwen2.5-Math/Coder-7B-Instruct) as an additional reward signal. This difference in supervision is an acknowledged advantage of dUltra and is explicitly modeled as a reward term rather than a training label; all methods still use the same base model (LLaDA-8B-Instruct) and are evaluated under identical generation settings. A detailed comparison of supervision signals is provided in Appendix E.

4.1 Training Dynamics

Figure 6 shows the evolution of reward metrics during dUltra training on GSM8K. Correctness, efficiency, and distillation rewards increase stably throughout training, validating the GRPO framework and multi-objective reward design. Detailed reward curves are included in Appendix H.

Effect of Advantage Clipping on Efficiency Reward. The role of advantage clipping in training stability is illustrated in Appendix H. Without clipping ($C = -\infty$), the planner collapses to a degenerate policy that never unmasks tokens; with $C = 0.0$, training remains stable and learns a balanced unmasking strategy.

4.2 Main Results

4.2.1 Results with Small Block Size

We first evaluate dUltra with block size $B = 32$, which provides a balance between sampling efficiency and unmasking selectivity. All tasks are evaluated in a zero-shot setting. Although the planner is trained as a Bernoulli policy, all reported inference results in Table 1 and Table 3 use deterministic threshold decoding: tokens are unmasked only when their planner probability exceeds the selected threshold. This removes sampling randomness and lets us control the accuracy-efficiency trade-off through the threshold. Table 1 presents our results on mathematical reasoning (GSM8K, MATH500) and code generation (HumanEval, MBPP) tasks, comparing against multiple accelerated MDLMs based on distillation, including dParallel and d3LLM. We report both task performance and the average number of function evaluations (NFE), i.e., the number of denoising steps during generation.

	Math				**Coding**			
	GSM8K		**MATH500**		**HumanEval**		**MBPP**	
Method	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)
Fast-dLLM	<u>77.56</u>	57.26 (4.33)	<u>35.00</u>	70.56 (3.52)	33.54	62.65 (3.94)	<u>37.80</u>	50.04 (4.16)
dParallel	77.10	28.47 (7.17)	31.80	<u>35.21</u> (5.32)	31.71	<u>30.68</u> (5.87)	36.20	<u>22.77</u> (6.30)
d3LLM	72.50	<u>25.79</u> (<u>8.39</u>)	33.80	35.57 (<u>6.13</u>)	<u>37.20</u>	33.41 (<u>6.86</u>)	33.20	32.26 (<u>6.90</u>)
dUltra (ours)	**81.29**	**21.21** (**9.72**)	**35.52**	**33.20** (**7.03**)	**39.02**	**24.25** (**7.23**)	**38.40**	**20.03** (**7.29**)

Table 1: Main results with generation length 256 and block size $B = 32$. dUltra rows use deterministic threshold decoding at inference time. We report accuracy, NFE, and T/F (tokens per model forward pass); **bold** and <u>underlined</u> values mark the best and second-best result in each column.

dUltra achieves strong performance across all benchmarks while maintaining low NFE. Notably, dUltra outperforms both distillation baselines (d3LLM and dParallel) on GSM8K and MATH500 in accuracy, while using fewer denoising steps on GSM8K and comparable denoising steps on MATH500, consistent with our mode filtering hypothesis: on-policy RL can discover better modes than methods limited to off-policy teacher trajectories. We further ablate the learned unmasking order planner by replacing it with confidence-based sampling on the same trained checkpoints (w/o planner); the planner-based dUltra improves accuracy by +5.9 pp on GSM8K and +12–17 pp on code generation tasks (see Appendix K.1). This demonstrates the value of jointly training the base model and unmasking order planner for task-specific unmasking strategies.

4.2.2 Results with Large Block Size

To explore the impact of larger block sizes on sampling efficiency, we evaluate dUltra with $B = 128$ in Appendix I. Larger block sizes give the planner more freedom for longer-horizon planning by allowing it to select from a broader set of candidate positions at each denoising step, potentially enabling faster inference through increased parallelism. Appendix K provides the corresponding speed-quality frontiers.

4.2.3 Wall-Clock Throughput

To verify that NFE reductions translate to real latency gains, we measure wall-clock throughput on a single H200 GPU for 256-token generation with $B = 128$ and no KV-cache. dUltra achieves 3.0–3.4$\times$ speedup over the LLaDA baseline and $\sim$1.5$\times$ speedup over d3LLM while improving accuracy on both GSM8K and MATH500; the full throughput table is in Appendix J.

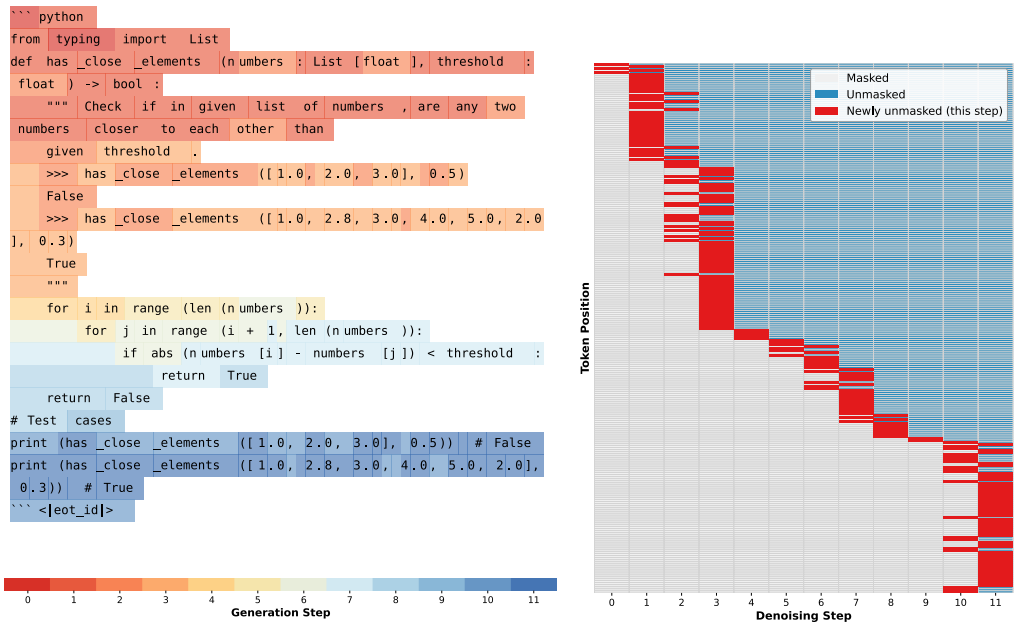

Figure 3: **Learned unmasking order on HumanEval.** Red bars indicate tokens newly unmasked at each denoising step. The planner mostly follows a left-to-right trend, but it also jumps to predictable code structure such as function signatures, control-flow keywords, and return statements, creating opportunities for parallel decoding while respecting stronger local dependencies.

4.3 Speed-Quality Trade-off

The deterministic threshold rule used for Table 1 and Table 3 also provides an inference-time speed-quality knob: increasing the threshold makes decoding more conservative, while decreasing it unmasks more tokens per step. We use Fast-dLLM's factor-based parallel decoding strategy [Wu et al., 2025] for models without a planner. Sweeping thresholds for dUltra and factors for baselines yields Pareto frontiers where dUltra consistently achieves superior accuracy-efficiency trade-offs; full frontier plots are provided in Appendix K.

4.4 Analysis of Learned Unmasking Strategies

To understand what unmasking order patterns dUltra learns, Figure 3 visualizes the unmasking behavior across denoising steps on a code-generation example; additional math and coding visualizations are provided in Appendix L. The learned policies are largely autoregressive, but code generation exhibits more non-autoregressive jumps to structural tokens such as function signatures, control-flow keywords, and return statements. These patterns support the main motivation: the planner selectively deviates from left-to-right order when task structure creates opportunities for parallel decoding while preserving quality. Extended related work is provided in Appendix B.

5 Conclusion

We present dUltra, a framework for learning adaptive unmasking strategies that accelerate masked diffusion language models through on-policy RL and distillation rewards. dUltra outperforms distillation baselines in both accuracy and NFE, and its learned policies adapt to task structure by exploiting more parallelism in code while preserving more sequential decoding in math. These results show that learned mode filtering and conditional-independence-aware planning can move MDLMs closer to practical parallel generation.

References

Lou Aaron. Language modeling by estimating the ratios of the data distribution. 2024. URL https://aaronlou.com/blog/2024/discrete-diffusion/.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces, 2021. URL https://arxiv.org/abs/2107.03006.

Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated Sampling from Masked Diffusion Models via Entropy Bounded Unmasking, May 2025. URL http://arxiv.org/abs/2505.24857. arXiv:2505.24857 [cs] version: 1.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dparallel: Learnable parallel decoding for dllms, 2025. URL https://arxiv.org/abs/2509.26488.

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. Dmax: Aggressive parallel decoding for dllms, 2026. URL https://arxiv.org/abs/2604.08302.

Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models, 2023. URL https://arxiv.org/abs/2210.08933.

Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation, 2025. URL https://arxiv.org/abs/2506.20639.

Google DeepMind. Gemini Diffusion is our new experimental research model. https://blog.google/technology/google-deepmind/gemini-diffusion/, May 2025. Accessed on July 25, 2025.

Ishaan Gulrajani and Tatsunori B. Hashimoto. Likelihood-based diffusion language models, 2023. URL https://arxiv.org/abs/2305.18619.

Zhengfu He, Tianxiang Sun, Kuanning Wang, Xuanjing Huang, and Xipeng Qiu. Diffusionbert: Improving generative masked language models with diffusion models, 2022. URL https://arxiv.org/abs/2211.15029.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html.

Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions, 2021. URL https://arxiv.org/abs/2102.05379.

Yanzhe Hu, Yijie Jin, Pengfei Liu, Kai Yu, and Zhijie Deng. Lightningrl: Breaking the accuracy-parallelism trade-off of block-wise dllms via reinforcement learning, 2026. URL https://arxiv.org/abs/2603.13319.

Zhanqiu Hu, Jian Meng, Yash Akhauri, Mohamed S. Abdelfattah, Jae sun Seo, Zhiru Zhang, and Udit Gupta. Accelerating diffusion language model inference via efficient kv caching and guided diffusion, 2025. URL https://arxiv.org/abs/2505.21467.

Zemin Huang, Zhiyang Chen, Zijun Wang, Tiancheng Li, and Guo-Jun Qi. Reinforcing the diffusion chain of lateral thought with diffusion language models, 2025. URL https://arxiv.org/abs/2505.10446.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report, 2024. URL https://arxiv.org/abs/2409.12186.

Daniel Israel, Guy Van den Broeck, and Aditya Grover. Accelerating diffusion llms via adaptive parallel decoding, 2025. URL https://arxiv.org/abs/2506.00413.

Metod Jazbec, Theo X. Olausson, Louis Béthune, Pierre Ablin, Michael Kirchhof, João Monteiro, Victor Turrisi, Jason Ramapuram, and Marco Cuturi. Learning unmasking policies for diffusion language models, 2025. URL https://arxiv.org/abs/2512.09106.

Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the Worst, Plan for the Best: Understanding Token Ordering in Masked Diffusions. June 2025. URL https://openreview.net/forum?id=DjJmre5IkP.

Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 4328–4343. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/1be5bc25d50895ee656b8c2d9eb89d6a-Paper-Conference.pdf.

Nianyi Lin, Jiajie Zhang, Lei Hou, and Juanzi Li. Boundary-guided policy optimization for memory-efficient rl of diffusion large language models. *arXiv preprint arXiv:2510.11683*, 2025.

Zhiyuan Liu, Yicun Yang, Yaojie Zhang, Junjie Chen, Chang Zou, Qingyuan Wei, Shaobo Wang, and Linfeng Zhang. dllm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025a.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025b.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution, June 2023. URL http://arxiv.org/abs/2310.16834. arXiv:2310.16834.

Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q. Weinberger. Latent diffusion for language generation, 2023. URL https://arxiv.org/abs/2212.09462.

Xinyin Ma, Runpeng Yu, Gongfan Fang, and Xinchao Wang. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*, 2025.

Chenlin Meng, Kristy Choi, Jiaming Song, and Stefano Ermon. Concrete score matching: Generalized score matching for discrete data. *Advances in Neural Information Processing Systems*, 35:34532–34545, 2022.

Tom Minka et al. Divergence measures and message passing. 2005.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL https://arxiv.org/abs/2502.09992.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data, 2025. URL https://arxiv.org/abs/2406.03736.

Fred Zhangzhi Peng, Zachary Bezemek, Sawan Patel, Jarrid Rector-Brooks, Sherwood Yao, Avishek Joey Bose, Alexander Tong, and Pranam Chatterjee. Path planning for masked diffusion model sampling, 2025. URL https://arxiv.org/abs/2502.03540.

Yu-Yang Qian, Junda Su, Lanxiang Hu, Peiyuan Zhang, Zhijie Deng, Peng Zhao, and Hao Zhang. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation. *ArXiv preprint*, to appear, 2025. URL https://github.com/hao-ai-lab/d3LLM.

Kevin Rojas, Jiahe Lin, Kashif Rasul, Anderson Schneider, Yuriy Nevmyvaka, Molei Tao, and Wei Deng. Improving reasoning for diffusion language models via group diffusion policy optimization. *arXiv preprint arXiv:2510.08554*, 2025.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models, June 2024. URL http://arxiv.org/abs/2406.07524. arXiv:2406.07524.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and Generalized Masked Diffusion for Discrete Data, June 2024. URL http://arxiv.org/abs/2406.04329. arXiv:2406.04329.

Andy Shih, Dorsa Sadigh, and Stefano Ermon. Training and inference on any-order autoregressive models the right way, 2022. URL https://arxiv.org/abs/2205.13554.

Xiaohang Tang, Rares Dolga, Sangwoong Yoon, and Ilija Bogunovic. wd1: Weighted Policy Optimization for Reasoning in Diffusion Language Models, July 2025. URL http://arxiv.org/abs/2507.08838. arXiv:2507.08838 [cs].

Benigno Uria, Iain Murray, and Hugo Larochelle. A deep and tractable density estimator, 2014. URL https://arxiv.org/abs/1310.1757.

Alex Wang and Kyunghyun Cho. Bert has a mouth, and it must speak: Bert as a markov random field language model, 2019. URL https://arxiv.org/abs/1902.04094.

Chenyu Wang, Paria Rashidinejad, DiJia Su, Song Jiang, Sid Wang, Siyan Zhao, Cai Zhou, Shannon Zejiang Shen, Feiyu Chen, Tommi Jaakkola, et al. Spg: Sandwiched policy gradient for masked diffusion language models. *arXiv preprint arXiv:2510.09541*, 2025a.

Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025b.

Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Dongrui Liu, and Linfeng Zhang. Accelerating diffusion large language models with slowfast sampling: The three golden principles, 2025. URL https://arxiv.org/abs/2506.10848.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025. URL https://arxiv.org/abs/2505.22618.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024. URL https://arxiv.org/abs/2409.12122.

Ling Yang, Ye Tian, Bowen Li, Xinchen Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. Mmada: Multimodal large diffusion language models, 2025. URL https://arxiv.org/abs/2505.15809.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL https://arxiv.org/abs/2508.15487.

Hanyang Zhao, Dawen Liang, Wenpin Tang, David Yao, and Nathan Kallus. Diffpo: Training diffusion llms to reason fast and furious via reinforcement learning, 2025a. URL https://arxiv.org/abs/2510.02212.

Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning, 2025b. URL https://arxiv.org/abs/2504.12216.

Haoyang Zheng, Xinyang Liu, Cindy Xiangrui Kong, Nan Jiang, Zheyuan Hu, Weijian Luo, Wei Deng, and Guang Lin. Ultra-fast language generation via discrete diffusion divergence instruct, 2025. URL https://arxiv.org/abs/2509.25035.

Yuchen Zhu, Wei Guo, Jaemoo Choi, Petr Molodyk, Bo Yuan, Molei Tao, and Yongxin Chen. Enhancing reasoning for diffusion llms via distribution matching policy optimization. *arXiv preprint arXiv:2510.08233*, 2025.

A Limitations

The on-policy distillation reward uses AR teacher models (Qwen2.5-Math/Coder-7B-Instruct); choosing domains where strong teachers exist is part of the experimental design rather than an inherent constraint of the framework. We do not include a head-to-head empirical comparison with the concurrent work of Jazbec et al. [2025] because their code and checkpoints were not public at the time of writing; an indirect comparison based on their reported numbers is given in Related Work. Throughputs in Table 4 are measured on a single hardware type (H200) with identical inference kernels across all methods.

B Related Work

Reinforcement Learning for MDLMs. Recent work has explored post-training diffusion language models (dLLMs) with reinforcement learning to improve their reasoning and generation capabilities. Early examples include `d1`, which adapts GRPO-style optimization to the (approximate) likelihood structure of dLLMs [Zhao et al., 2025b], and DiffuCoder, which targets code generation for LLaDA-style masked diffusion models [Gong et al., 2025]. A growing line of follow-up work focuses on improving the stability and bias/variance trade-offs of policy-gradient estimators for dLLMs [Tang et al., 2025, Rojas et al., 2025, Lin et al., 2025, Wang et al., 2025a, Zhu et al., 2025, Yang et al., 2025]. LightningRL [Hu et al., 2026] is especially related in its goal of improving the speed-quality frontier of block-wise dLLMs with RL. Its main reported results, however, use the SDAR backbone, whereas our experiments are built on LLaDA-8B-Instruct; including the LightningRL headline numbers in the main table would therefore conflate method differences with base-model differences. Methodologically, LightningRL samples rollouts with confidence-threshold remasking, whereas dUltra learns the token-level unmasking policy through an explicit planner head. Notably, Wang et al. [2025b], Zhao et al. [2025a] try to obtain more accurate likelihood by using more intermediate decoding steps. More broadly, prior RL work either improves reasoning capability or tunes speed-quality behavior around an existing sampler, while dUltra directly learns an adaptive token-level unmasking policy and enables high quality mode filtering via RL. Close to our work is DCOLT [Huang et al., 2025], which also learns an unmasking order policy with RL. However, their unmasking policy does not enable parallel decoding due to the adoption of the Plackett-Luce model. Zhao et al. [2025a] learns adaptive confidence thresholds for unmasking with RL, but their method still relies on logit-based heuristics for token selection rather than learning a flexible unmasking policy as in our work. Concurrent work Jazbec et al. [2025] also learns an unmasking policy head with GRPO, but there are several important distinctions. First, they learn the unmasking policy from the logits rather than the last-layer representation. Second, they keep the base model fixed rather than jointly training it with the unmasking policy. Third, they do not introduce an on-policy distillation reward to encourage mode-seeking behavior. As a result, they report only marginal benefits over logit-based heuristics, while we achieve better performance in both accuracy and NFE even when compared to SOTA distillation-based methods.

MDLM Acceleration. Most existing acceleration methods for masked dLLMs are either system level, such as enabling KV cache and employing token dropping [Ma et al., 2025, Liu et al., 2025a, Hu et al., 2025, Wu et al., 2025], or algorithmic level methods that design better sampling strategies. Among the latter, there are two approaches: (i) training-free logit-based heuristic decoders [Wu et al., 2025, Israel et al., 2025, Wei et al., 2025, Ben-Hamu et al., 2025] and (ii) *distillation*-based approaches that fit off-policy trajectories produced by a base model [Chen et al., 2025, Qian et al., 2025]. Heuristic samplers often rely on confidence, entropy, or related uncertainty signals computed from logits, and have recently enabled substantial inference speedups in dLLMs via parallel decoding and KV caching [Wu et al., 2025, Hu et al., 2025]. Distillation-based methods, such as pseudo-trajectory distillation in d3LLM [Qian et al., 2025], learnable parallel decoding in dParallel [Chen et al., 2025], and distribution-matching distillation in DiDi-Instruct [Zheng et al., 2025], can further increase decoding efficiency by fine-tuning the base model to sample multiple steps of the base model in one step, which essentially filters the modes of the base model. DMax [Chen et al., 2026] provides another recent acceleration route by reformulating masked decoding as progressive self-refinement with soft parallel decoding. These methods depend on the quality and coverage of the teacher trajectories, specialized training distribution, or refinement dynamics, which may become off-policy as training progresses. Our approach explicitly addresses this mismatch by using on-policy rollouts with GRPO, while also encouraging mode-seeking behavior.

Denoising Trajectory Planning. Beyond heuristic confidence-based planning, recent works have explored learned planning modules for MDLM trajectories. These approaches differ fundamentally in their planning strategies. P2 [Peng et al., 2025] introduces explicit planning and denoising sub-stages, enabling iterative refinement of existing unmasked tokens. DCOLT [Huang et al., 2025] employs the Plackett-Luce model to generate sequential unmasking orders optimized for accuracy rather than parallel decoding efficiency. DiFFPO [Zhao et al., 2025a] learns adaptive per-prompt thresholds via off-policy RL but still selects tokens based on confidence heuristics. Concurrently, Jazbec et al. [2025] learns an unmasking policy head with GRPO on token confidences from logits.

C Mode Filtering Illustration

Parallel decoding can expose unwanted modes when several dependent ambiguous tokens are sampled independently in the same denoising step. Figure 4 illustrates this phenomenon and motivates learning unmasking trajectories that decode only compatible sets of tokens in parallel.

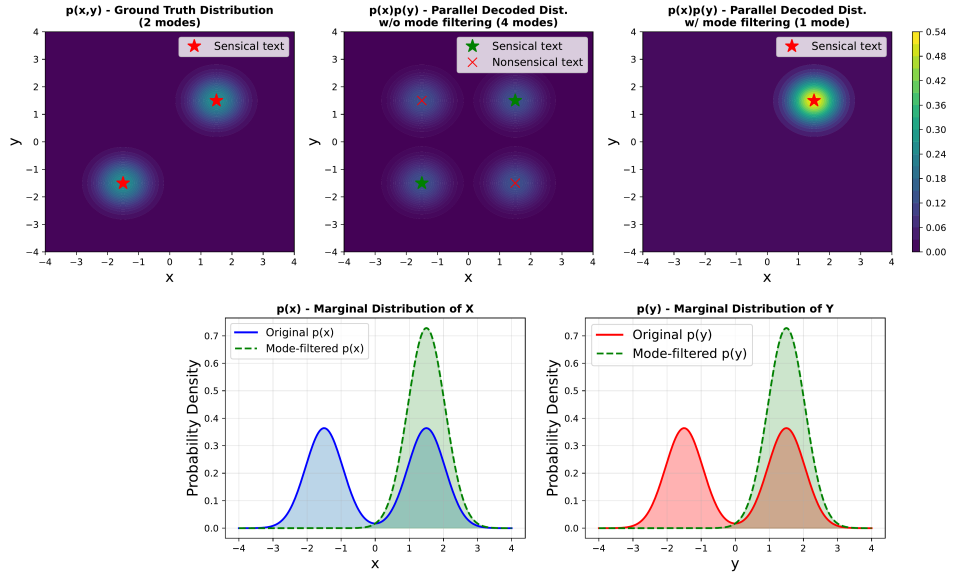

Figure 4: Parallel decoding of dependent ambiguous tokens can lead to unwanted modes.

D Motivating Case Study

To illustrate the potential of conditional independence-aware planning, we compare the quality of one-step generation of LLaDA-Instruct under different masking strategies in Figure 5. More specifically, we compare random masking, where we randomly select a certain percentage of tokens in the ground truth to mask, and autoregressive masking, where we mask the same percentage of tokens only at the end of the ground truth sequence. We observe that LLaDA-Instruct is able to decode a large percentage (30%) of masked tokens with no noticeable quality degradation with random masking, while masking the same percentage of mask tokens at the end of the sequence leads to poor generation quality. This is expected as randomly masked tokens usually have more contextual information making them less conditionally dependent on other masked tokens, while consecutive tokens at the end of the sequence are highly dependent on each other, even if given the partial answer.

E Baseline Supervision Comparison

Table 2 summarizes the supervision and training requirements of each method.

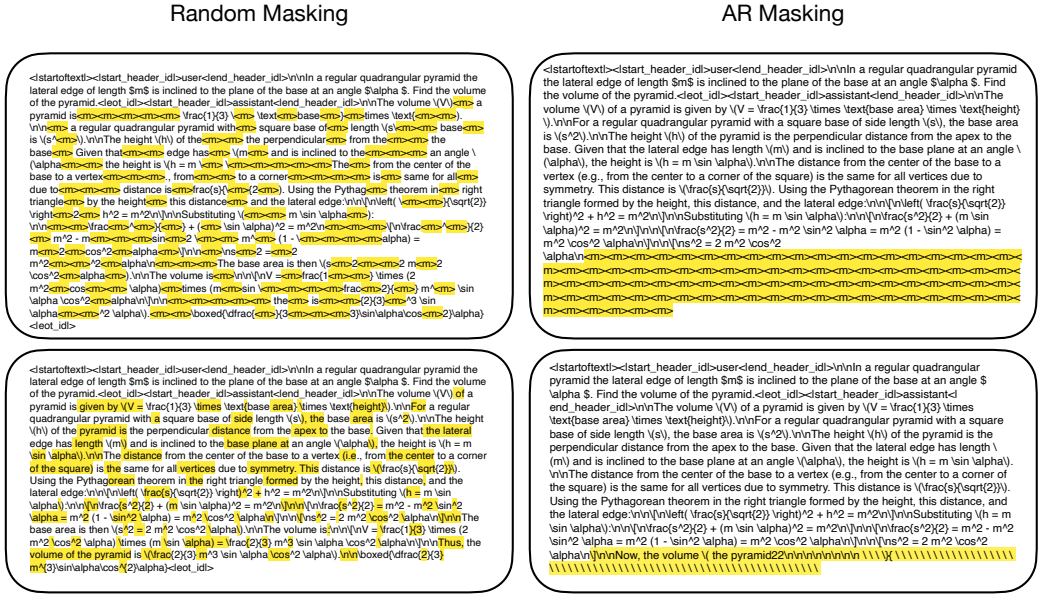

Figure 5: Given the same prompt and ground truth, we mask 30% of tokens using both random masking and autoregressive (AR) masking. The left column shows the masked input sequence with random masking (<m> represents the mask token), and the corresponding one-step decoding result. The right columns show the masked input sequence with AR masking, and the corresponding one-step decoding result. We use `LLaDA-8B-Instruct` for both generations.

Method	Training	Teacher signal	Rollout policy	Base model updated	Planner learned
Fast-dLLM	None	None (logit heuristic)	—	No	No
dParallel	SFT	Self (base model)	Off-policy	Yes	No
d3LLM	SFT	Self (base model)	Off-policy	Yes	No
dUltra	GRPO	AR teacher (Qwen2.5)	On-policy	Yes	Yes

Table 2: Comparison of supervision and training requirements across acceleration methods. dUltra is the only method that (i) learns an explicit unmasking planner, (ii) trains with on-policy rollouts, and (iii) uses an external AR teacher as a reward signal. This additional supervision is an intentional design choice that enables mode-seeking behavior beyond what self-distillation can provide.

F Planner Initialization

We find that if we initialize the unmasking planner head randomly, the expected number of tokens unmasked at each denoising step $n = \sum_i p_{\text{unmask}}(x_i)$ can be excessively large, leading to degenerate performance at the onset of training and preventing RL from effectively exploring trajectories with more sampling steps and better performance. We therefore initialize the planner before applying RL so that the RL algorithm uses a heuristic logit-based planner as a baseline to optimize the sampling order.

More specifically, we fine-tune the unmasking planner head using supervised learning to mimic the confidence-aware parallel decoding method proposed in Fast-dLLM [Wu et al., 2025]. We formulate this planner fine-tuning as a binary classification problem: given a partially masked input, the planner predicts which tokens should be unmasked at the current step according to the Fast-dLLM confidence criterion. For a set of masked positions $\mathcal{I}_{\text{block}}$, we optimize the unmasking planner head ϕ using binary cross-entropy loss with position weighting to handle class imbalance:

$$\mathcal{L}_{\text{planner}} = \frac{1}{|\mathcal{I}_{\text{block}}|} \sum_{i \in \mathcal{I}_{\text{block}}} \text{BCE}\big(\phi_i(\boldsymbol{h}), y_i; w_{\text{pos}} = \frac{\#\text{negatives}}{\#\text{positives}}\big), \tag{10}$$

where $\phi_i(\boldsymbol{h})$ is the predicted unmasking probability for the i-th masked token, $y_i \in \{0, 1\}$ indicates whether token i should be unmasked according to Fast-dLLM, and w_{pos} weights tokens that should be unmasked to compensate for class imbalance.

G Advantage Clipping Details

Vanilla GRPO can lead to the degenerate solution where all unmasking probabilities collapse to 0, even with the exact likelihood computed in Section 3.2. While this gives high distillation reward due to extremely slow sampling, it defeats the purpose of acceleration. We hypothesize that this occurs because trajectories with low advantage may be overly conservative: if the model could have unmasked 5 tokens but the planner only unmasked 3, the trajectory receives low advantage due to inefficiency. Penalizing this trajectory uniformly reduces the likelihood of all unmasking decisions, including the 3 tokens that were correctly unmasked.

Advantage clipping avoids this failure mode by setting below-threshold advantages to zero, focusing updates on successful trajectories. This is useful because the space of possible unmasking orders is exponentially large compared to valid high-quality unmasking orders, making it more effective to exploit successful discoveries than to learn uniformly from failed attempts. In our experiments, setting $C = 0$ stabilizes training.

H Training Dynamics and Stabilization

Figure 6 shows the evolution of reward metrics during dUltra training on GSM8K. The correctness reward is measured by the correctness of generated solutions, and receives 2.0 if the answer is correct and 0.0 otherwise. The efficiency reward is defined as the negative number of denoising steps (NFE) times a constant. The distillation reward is the log likelihood of rollouts with the teacher model.

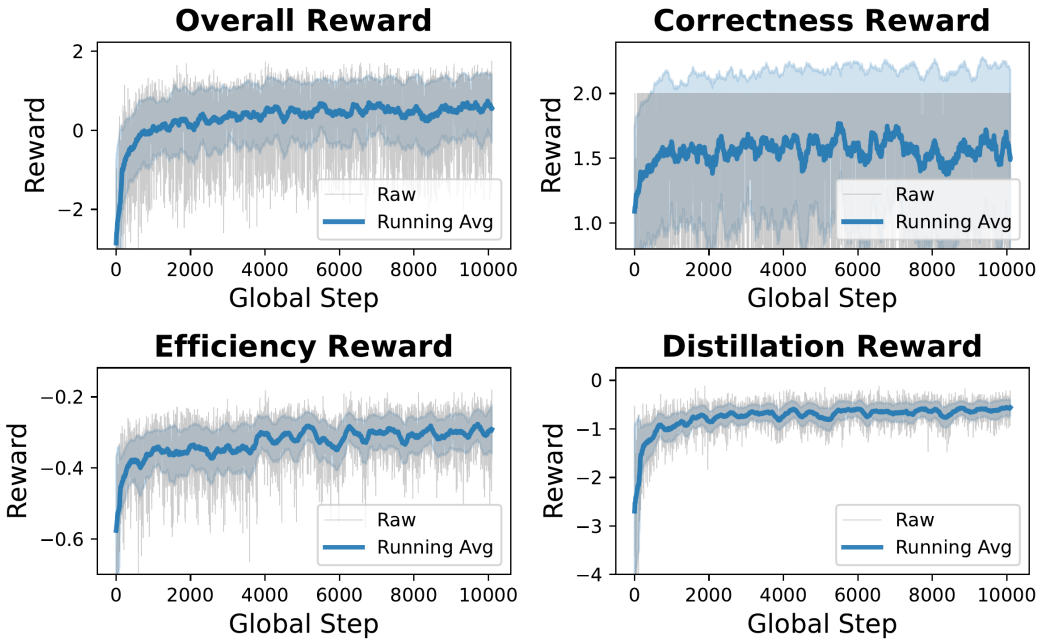

Figure 6: Training dynamics on GSM8K.

Figure 7 illustrates the role of advantage clipping in training stability. Without advantage clipping ($C = -\infty$), the training quickly becomes unstable: the average NFE diverges to extreme values as the learned planner collapses to a degenerate policy that never unmasks tokens. This occurs because negative advantages from poor trajectories can dominate the gradient signal, causing the planner to become overly conservative. With appropriate advantage clipping ($C = 0.0$), training remains stable and the model learns a balanced unmasking strategy. The clipping mechanism ensures that

only rollouts with above-average returns contribute to policy updates, preventing the model from over-fitting to failure modes.

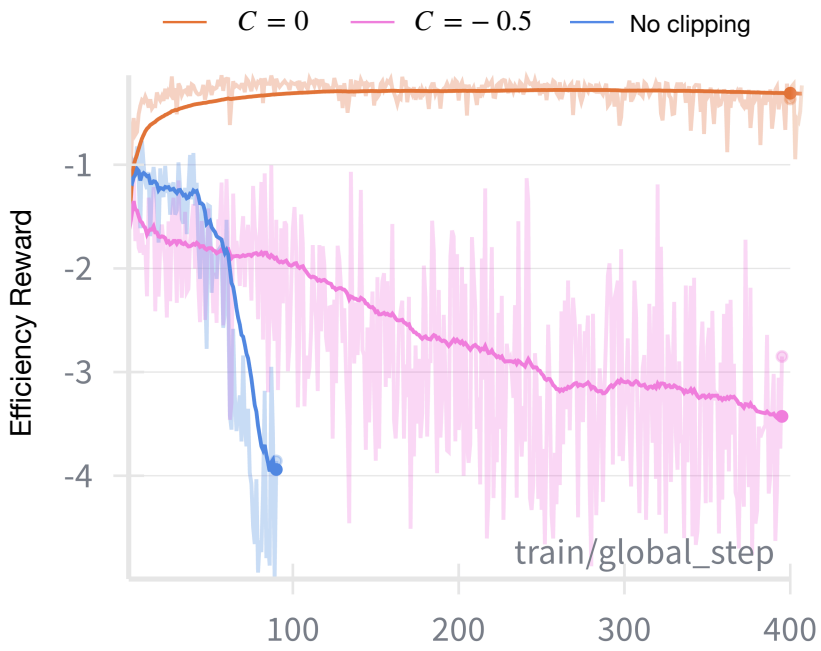

Figure 7: Effect of advantage clipping value C on training stability. Without advantage clipping, the average NFE quickly diverges as the planner collapses to never unmasking any tokens. With advantage clipping, training remains stable and the model learns to reduce NFE while maintaining task performance.

I Additional Main Results with Large Block Size

Table 3 presents the full results with block size 128 with all other settings the same as results with block size 32. As in Table 1, all dUltra inference results use deterministic threshold decoding rather than random Bernoulli sampling. Larger block sizes enable faster sampling with minimal accuracy loss; dUltra achieves 16.38 NFE on GSM8K (23% reduction vs. $B = 32$) with only 3.6 pp accuracy drop.

	Math				Coding			
	GSM8K		MATH500		HumanEval		MBPP	
Method	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)
Fast-dLLM	<u>75.66</u>	54.43 (4.56)	<u>32.60</u>	75.72 (3.30)	<u>35.40</u>	61.54 (4.08)	<u>37.80</u>	56.93 (4.07)
dParallel	<u>75.66</u>	24.07 (8.23)	31.40	28.49 (5.45)	30.49	<u>23.68</u> (6.26)	36.00	<u>18.07</u> (7.63)
d3LLM	69.07	<u>21.37</u> (<u>9.59</u>)	31.40	<u>26.65</u> (<u>7.03</u>)	32.32	24.49 (<u>7.34</u>)	33.80	26.37 (<u>7.66</u>)
dUltra (ours)	**77.65**	**16.38** (**13.96**)	**33.64**	**26.53** (**9.25**)	**36.83**	**21.79** (**9.43**)	**39.12**	**14.78** (**9.97**)

Table 3: Main results with generation length 256 and block size $B = 128$. dUltra uses deterministic threshold decoding at inference time. We report accuracy, NFE, and T/F (tokens per model forward pass); **bold** and <u>underlined</u> numeric values mark the best and second-best results for each metric.

J Wall-Clock Throughput

To verify that NFE reductions translate to real latency gains, we measure wall-clock throughput (tokens/second) on a single H200 GPU for 256-token generation with block size $B = 128$. We

compare against the LLaDA baseline (Fast-dLLM) and distillation-based acceleration methods. No KV-cache is used.

	GSM8K			MATH500		
Method	Acc↑	Tok/s↑	Speedup↑	Acc↑	Tok/s↑	Speedup↑
LLaDA baseline	<u>75.7%</u>	252	1.0×	<u>32.6%</u>	176	1.0×
dParallel	<u>75.7%</u>	459	1.8×	31.4%	298	1.7×
d3LLM	69.1%	<u>532</u>	<u>2.1×</u>	31.4%	<u>378</u>	<u>2.1×</u>
dUltra (ours)	**77.7%**	**851**	**3.4×**	**32.8%**	**524**	**3.0×**

Table 4: Wall-clock throughput on a single H200 GPU (256-token generation, $B = 128$). dUltra achieves 3.0–3.4× speedup over the LLaDA baseline and ∼1.5× over the best distillation baseline (d3LLM), while simultaneously improving accuracy on both tasks. **Bold** and <u>underlined</u> values mark the best and second-best results for each metric; tied second-best values are both underlined.

K Pareto Frontiers

At inference time, we eliminate Bernoulli sampling randomness by deterministically unmasking only tokens whose planner probability exceeds a threshold; this is the rule used for both Table 1 and Table 3. We use Fast-dLLM's factor-based parallel decoding strategy [Wu et al., 2025] for models without a planner. We sweep planner thresholds for dUltra models with planners and Fast-dLLM factors for dUltra without planners, and show the resulting Pareto frontiers in Figure 8. No-planner ablations replace the learned planner with confidence-based decoding while keeping the base model fixed.

K.1 Ablation Results Without the Planner

Table 5 reports the no-planner ablation obtained by removing the learned planner from dUltra and decoding with confidence-based sampling. These rows quantify the accuracy, NFE, and T/F trade-offs when the planner is replaced by a fixed confidence-based decoding rule.

	Math				Coding			
	GSM8K		MATH500		HumanEval		MBPP	
Method	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)	Acc↑	NFE↓ (T/F↑)
dUltra w/o planner ($B = 32$)	75.44	15.31 (13.39)	34.00	24.67 (9.45)	23.78	16.18 (10.19)	20.40	12.18 (11.28)
dUltra w/o planner ($B = 128$)	69.67	10.46 (22.16)	32.80	19.36 (12.80)	21.95	13.18 (15.00)	21.60	8.27 (17.65)

Table 5: No-planner ablation for dUltra. Each row removes the learned planner and uses confidence-based decoding, reporting the benchmark-matched dUltra result for each task group.

L Learned Unmasking Strategies

To understand what unmasking order patterns dUltra learns, we visualize the unmasking behavior across denoising steps on both math and coding tasks (Figure 9 and Figure 10). Each heatmap shows when tokens are unmasked during the denoising process, with red bars indicating newly unmasked tokens at each step. The vertical axis represents token positions, while the horizontal axis shows denoising steps. For both visualizations, we use a block size of 128 to allow more flexible unmasking orders.

The visualizations reveal that the major trend across both tasks is largely autoregressive: tokens are predominantly unmasked in left-to-right order, as evidenced by the diagonal progression from top-left to bottom-right in both heatmaps. This suggests that despite the theoretical flexibility of masked diffusion models, the autoregressive order often serves as a good enough prior. This also explains the success of AR models and speculative decoding in practice.

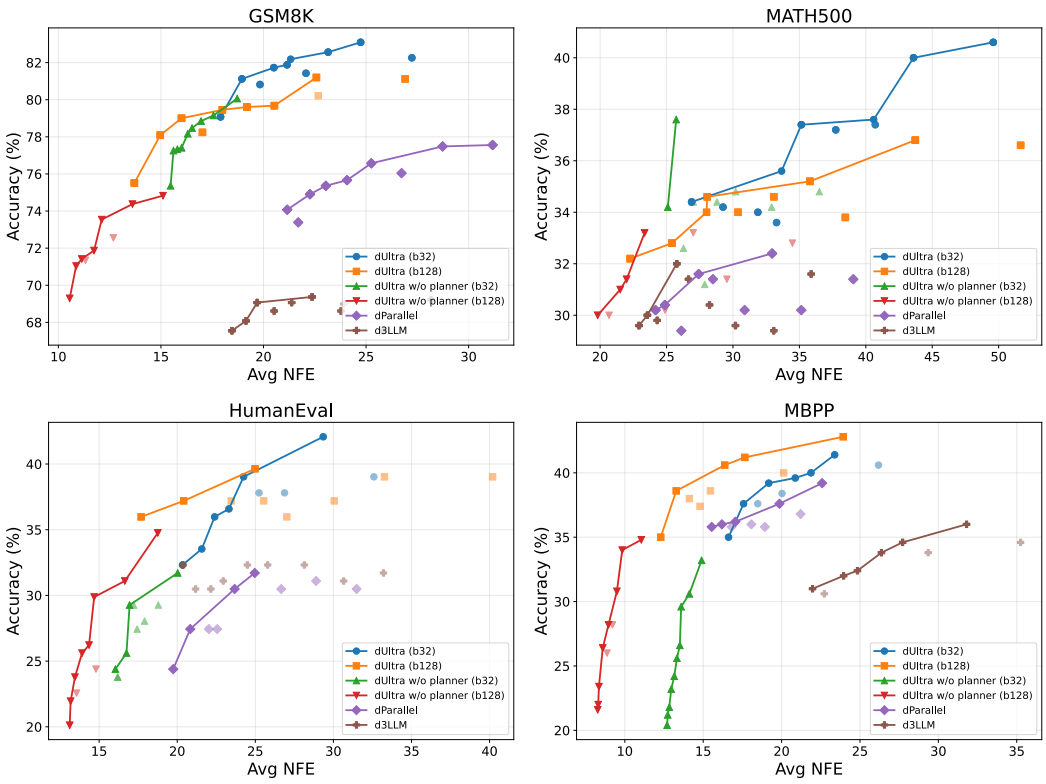

Figure 8: Accuracy-efficiency Pareto frontiers across math reasoning and code generation benchmarks. We compare dUltra against baseline methods on GSM8K (top-left), MATH500 (top-right), HumanEval (bottom-left), and MBPP (bottom-right). The x-axis shows average NFE (lower is faster), while the y-axis shows task accuracy. Each point represents a different threshold for dUltra and a different factor for models without planners. The no-planner dUltra ablations directly use the output logits for confidence-based token selection. dUltra consistently achieves strong accuracy-efficiency trade-offs compared to distillation-based methods, demonstrating the effectiveness of our learned unmasking strategies via on-policy distillation.

Nevertheless, we observe that code generation exhibits more non-autoregressive behavior compared to mathematical reasoning. In Figure 10, we observe visible jumps where the model unmasks function signatures (`def has_close_elements`), control flow keywords (`for`, `if`), and return statements earlier than strict left-to-right order would dictate. These structural elements provide crucial context for generating subsequent code and can be predicted with high confidence from the prompt alone. The model exploits this hierarchical dependency structure to achieve greater parallelism: by unmasking the skeleton of the code early, it creates multiple independent sub-problems (filling in loop bodies, conditions, etc.) that can be decoded in parallel.

In contrast, mathematical reasoning in Figure 9 follows a smoother, more sequential progression. While there are minor deviations—such as unmasking numbers or operators that can be inferred from the problem statement—the overall pattern adheres closely to autoregressive order. This reflects the fundamentally sequential nature of step-by-step reasoning: each calculation step typically depends on previous results, limiting opportunities for parallel decoding.

Overall, these unmasking patterns also validate our motivation: the learned planner creates masking distributions that locally resemble the random masking strategy from our case study (Figure 5), selectively deviating from autoregressive order to unmask conditionally independent tokens in parallel while maintaining quality. The degree of non-autoregressive behavior adapts to task structure, with code exploiting more parallelism opportunities than sequential mathematical reasoning. Since consecutive tokens are usually highly dependent on each other in the true data distribution, the model's ability to generate many consecutive tokens in parallel demonstrates the mode filtering

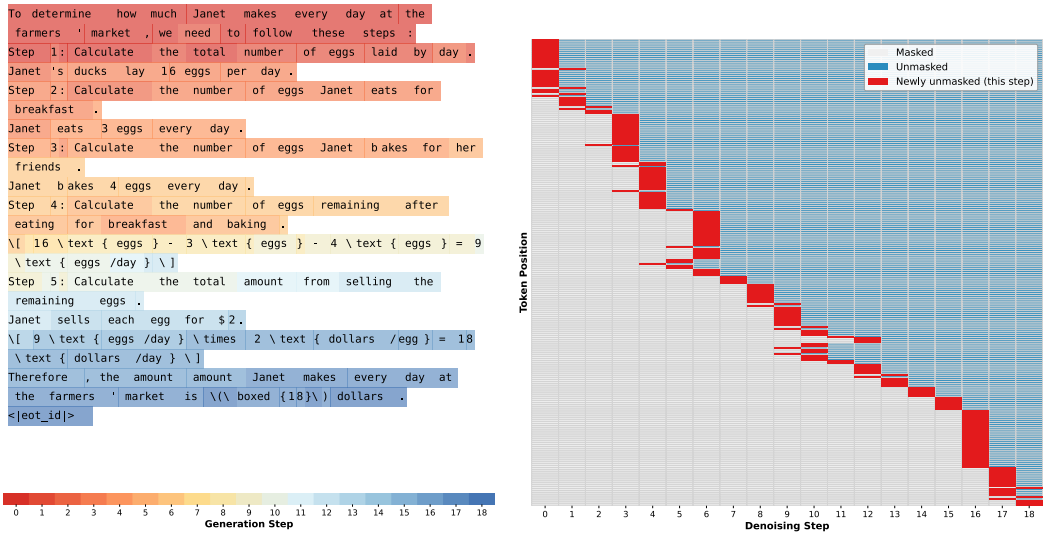

Figure 9: Visualization of learned unmasking patterns on GSM8K. Red bars show newly unmasked tokens at each denoising step. The pattern exhibits a predominantly autoregressive (left-to-right) trend with minor deviations. An interesting observation is that sometimes, "Step" and ":" are generated first without the numbering in between, and the numbering is generated in the next step.

behavior discussed in the introduction: parallel generation of dependent tokens independently can be realized by filtering the modes of the joint distribution, trading diversity for speed.

M MDLM Background

M.1 Single-Token Forward and Reverse Process

Continuous-time MDLMs [Shi et al., 2024, Sahoo et al., 2024] define a forward noising process for $t, s \in [0, 1]$ with $t > s$:

$$\begin{aligned} q_{t|s}(x_t|x_s = m) &= \mathrm{Cat}(\boldsymbol{e}_m), \\ q_{t|s}(x_t|x_s = x_0) &= \mathrm{Cat}\left(\frac{\alpha_t}{\alpha_s}\boldsymbol{e}_{x_0} + \frac{\alpha_s - \alpha_t}{\alpha_s}\boldsymbol{e}_m\right), \end{aligned} \tag{11}$$

where α_t is the noise schedule with $\alpha_0 = 1$ and $\alpha_1 = 0$. The marginal is $q_{t|0}(x_t|x_0) = \mathrm{Cat}(\alpha_t \boldsymbol{e}_{x_0} + (1 - \alpha_t)\boldsymbol{e}_m)$. The time reversal for $s < t$ given x_0 is

$$q_{s|t,0}(x_s|x_t, x_0) = \begin{cases} \mathrm{Cat}(\boldsymbol{e}_{x_0}), & x_t = x_0 \\ \mathrm{Cat}\left(\frac{\alpha_s - \alpha_t}{1-\alpha_t}\boldsymbol{e}_{x_0} + \frac{1-\alpha_s}{1-\alpha_t}\boldsymbol{e}_m\right) & x_t = m. \end{cases} \tag{12}$$

Similar to DDPM [Ho et al., 2020], the model $\boldsymbol{\mu}_\theta$ predicts the clean sample x_0:

$$p_{\boldsymbol{\theta}}(x_s|x_t) = q_{s|t,0}(x_s|x_t, \boldsymbol{e}_{x_0} \approx \boldsymbol{\mu}_\theta(x_t, t)). \tag{13}$$

M.2 NELBO Derivation

The approximation in Equation (13) is optimized by minimizing the NELBO. Given discretization steps T, with $s(i) = (i-1)/T$ and $t(i) = i/T$, the NELBO decomposes as:

$$\mathcal{L}_{\text{NELBO}} = \mathbb{E}_q \Bigg[\underbrace{-\log p_\theta(x \mid x_{t(1)})}_{\mathcal{L}_{\text{recons}}} + \underbrace{\sum_{i=2}^{T} D_{\text{KL}}\big[q(x_{s(i)} \mid x_{t(i)}, x) \,\|\, p_\theta(x_{s(i)} \mid x_{t(i)})\big]}_{\mathcal{L}_{\text{diffusion}}} + \underbrace{D_{\text{KL}}\big[q(x_{t(T)} \mid x) \,\|\, p_\theta(x_{t(T)})\big]}_{\mathcal{L}_{\text{prior}}} \Bigg]. \tag{14}$$

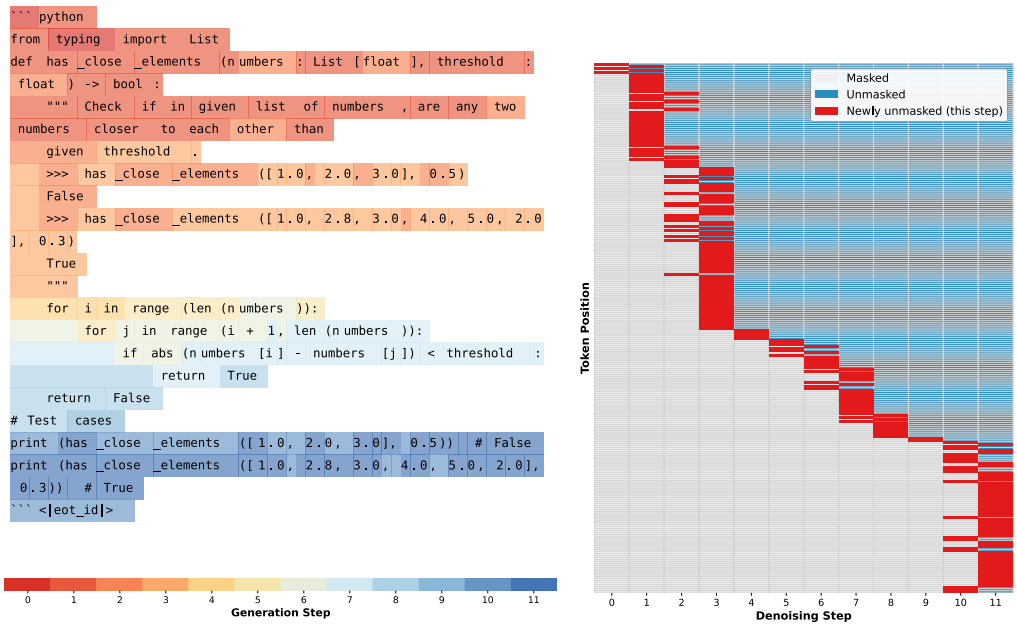

Figure 10: Visualization of learned unmasking patterns on HumanEval. While the overall trend remains autoregressive, code generation exhibits more non-autoregressive behavior with visible jumps to unmask function signatures, control flow keywords, and structural elements earlier than strict left-to-right order.

For MDLMs with carry-over unmasking [Sahoo et al., 2024, Shi et al., 2024], this simplifies to the weighted cross-entropy in Eq. 1 of the main paper.

N Experimental Setup

N.1 Implementation Details

N.1.1 Model Architecture

Our model extends the LLaDA-Instruct model [Nie et al., 2025] with an unmasking planner head. The architecture consists of:

Base Model. We use LLaDA [Nie et al., 2025], a masked diffusion language model with Llama-like architecture adapted for discrete diffusion. The model uses a special mask token and employs time-agnostic parametrization following recent findings that the optimal ELBO solution is independent of the time variable [Ou et al., 2025, Sahoo et al., 2024].

Unmasking Planner Head. Following the architecture in Huang et al. [2025], we introduce an unmasking planner head that takes the last-layer hidden states and outputs per-token unmasking probabilities:

$$p_{\text{unmask}}(x) = \text{sigmoid}(\text{head}(h_x)) \tag{15}$$

where h_x is the hidden state for token position x. The head consists of:

- $L = 1$ transformer blocks (LLaDABlock layers)
- Adaptive Layer Normalization (AdaLN) with timestep conditioning
- Mask embeddings
- Final linear projection to scalar logit

Since the unmasking planner head is small (a single transformer block), it adds only a small computational overhead relative to the base model.

N.1.2 Training Configuration:

- Unmasking planner head fine-tuning: Learning rate 1e-6, batch size 4, cosine schedule with warmup
- GRPO: Block length 32/128, temperature 0.1, advantage clipping, group size 12, learning rate 5e-6
- Optimizer: AdamW with weight decay 0.01
- Hardware: Single H200 GPU

N.2 Reward Function

Our training uses a multi-component reward function that combines task-specific correctness verification, format compliance, and efficiency incentives. The total reward is computed as a weighted sum:

$$r_{\text{total}} = \sum_i \lambda_i \cdot r_i \tag{16}$$

where r_i are individual reward components and λ_i are their weights. Below we detail the default configuration for mathematical reasoning and code generation tasks.

N.2.1 Mathematical Reasoning Tasks (GSM8K, MATH500)

Correctness Reward ($\lambda = 1.0$): Extracts the final answer from LaTeX \boxed{} notation and verifies mathematical equivalence against ground truth using `math_verify` library for symbolic comparison. Returns 2.0 for correct answers, 0.0 otherwise.

Format Reward ($\lambda = 1.0$): Rewards proper use of LaTeX boxed notation (\boxed{answer}) for presenting the final answer. Returns 0.5 when answers are properly boxed, 0.0 otherwise. This encourages the model to follow standard mathematical writing conventions.

Efficiency Reward ($\lambda = 1.0$): Incentivizes faster sampling by penalizing the number of denoising steps: $r_{\text{step}} = -N_{\text{steps}}/50$, where N_{steps} is the total number of denoising iterations.

On-Policy Distillation Reward ($\lambda = 1.0$): Computes the average per-token log-likelihood of the generated completion under `Qwen2.5-Math-7B-Instruct` teacher model: $r_{\text{distill}} = \frac{1}{L}\sum_{i=1}^{L} \log p_{\text{teacher}}(c_i \mid \boldsymbol{q}, \boldsymbol{c}_{<i})$, where L is the completion length. This encourages mode-seeking behavior by rewarding completions that the teacher assigns high probability.

N.2.2 Code Generation Tasks (HumanEval, MBPP)

Correctness Reward ($\lambda = 3.0$): Executes the generated code against test cases with a 10-second timeout per test. Returns 1.0 if all tests pass, 0.0 for any wrong outputs, and -0.05 for runtime errors (excluding syntax/indentation errors which return 0.0). Code is extracted from markdown fences and wrapped to support LeetCode-style class solutions.

Code Fence Format Reward ($\lambda = 1.0$): Encourages proper markdown code formatting with incremental scoring: +0.25 for any fence, +0.25 for ```` ```python ```` fence, +0.25 for closing fence, +0.25 for no trailing text. Small penalty (0.001 per character) for extra content after closing fence. Maximum score: 1.0.

Efficiency Reward ($\lambda = 0.1$): Same as math tasks.

On-Policy Distillation Reward ($\lambda = 1.0$): Uses `Qwen2.5-Coder-7B-Instruct` teacher model. Computation is identical to math tasks but with domain-specific teacher.

O Training Algorithm

Algorithm 1: Accelerating diffusion LLM with unmasking order planning using GRPO

Input: diffusion LLM model θ, unmasking planner head ϕ, dataset $\mathcal{D}$, reward_func, advantage clipping threshold C

while *θ and ϕ not converged and maximum epochs not reached* **do**

  Sample questions $q \sim \mathcal{D}$

  for $g \leftarrow 1$ **to** G **do**

    `// Generate a group of` G `trajectories`

    Initialize x^g with q and mask tokens

    $k \leftarrow 0$ `// number of steps`

    while x^g *has mask tokens* **do**

      Run the model forward once to get the logits l and last-layer hidden state h

      Calculate unmasking probability $\pi_{\text{unmask}} = \text{sigmoid}(\phi(h))$ for each mask token

      Calculate token probability $\pi_{\text{token}} = \text{softmax}(l)$

      Sample the mask tokens to unmask $\mathbf{Pos}^{(k)} \sim \text{MultivariateBernoulli}(\pi_{\text{unmask}})$

      Sample the value of unmasked tokens $\mathbf{x}^g_{\mathbf{Pos}^{(k)}} \sim \pi_{\text{token}}$

      $k \leftarrow k + 1$

    $k^g \leftarrow k$

    $r^g \leftarrow \text{reward\_func}(x^g)$ `// Compute the rewards`

  for $g \leftarrow 1$ **to** G **do**

    $A^g \leftarrow r^g - \text{mean}(r^{1:G})$ `// Compute advantages`

    if $A^g < C$ **then**

      $A^g \leftarrow 0$ `// Advantage clipping`

  $N \leftarrow \max(k^g)$

  `// Loop through the group`

  for $g \leftarrow 1$ **to** G **do**

    `// Loop through denoising steps`

    for $n \leftarrow 1$ **to** N **do**

      if $k^g > n$ **then**

        `// Compute` π_θ `and losses`

        $\pi\left(\boldsymbol{x}^{g,n} \mid \boldsymbol{x}^{g,n-1}\right) = \pi_{\text{select}}\left(\mathbf{Pos}^{(n)} \mid \boldsymbol{x}^{g,n-1}\right) \cdot \pi_{\text{token}}\left(\boldsymbol{x}^{g,n} \mid \boldsymbol{x}^{g,n-1}, \mathbf{Pos}^{(n)}\right)$ `// Equation (3)`

        $\mathcal{L}_{\theta,n} \leftarrow -\frac{1}{|\boldsymbol{x}|\cdot G} \sum_{g=1}^{G} \frac{\pi(\boldsymbol{x}^{g,n}|\boldsymbol{x}^{g,n-1})}{\text{sg}(\pi(\boldsymbol{x}^{g,n}|\boldsymbol{x}^{g,n-1}))} A^g$ `// sg is stop gradient`

        Accumulate gradient $\nabla_\theta \mathcal{L}_{\theta,n}$

  Update θ with accumulated gradients $\sum_{n=1}^{N} \nabla_\theta \mathcal{L}_{\theta,n}$

  Zero out accumulated gradients

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the sampling-efficiency bottleneck for MDLMs, the proposed dUltra framework, the learned planner and GRPO training contributions, and the evaluated math and coding benchmarks.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper includes a Limitations section discussing reliance on AR teacher models for the on-policy distillation reward, the domain dependence introduced by requiring strong teacher models, the lack of a head-to-head comparison with concurrent work whose code and checkpoints were not public at the time of writing, and the single-H200 hardware context for throughput measurements.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present formal theorem statements or proof-based theoretical results. It includes mathematical derivations and likelihood factorizations, with assumptions such as independent Bernoulli unmasking decisions stated in the methods and appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper and appendix describe the model architecture, planner initialization, GRPO objective, reward functions, evaluation benchmarks, generation length, block sizes, deterministic threshold decoding, training hyperparameters, and evaluation metrics. To preserve anonymity, code and checkpoints are not linked in the submission, but the paper states that they will be released upon acceptance.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: Code and checkpoints will be released upon acceptance and provides implementation and evaluation details in the main text and appendix.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The main text and appendix specify the base model, block sizes, generation length, deterministic inference rule, baselines, teacher models, planner architecture, optimizer, learning rates, batch and group sizes, temperature, reward weights, and task-specific evaluation procedures.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper reports deterministic benchmark accuracies, NFE, TPF, Pareto frontiers, and throughput, but it does not report error bars, confidence intervals, or statistical significance tests for the main experimental results, since during inference we threshold the planner unmasking probability and a temperature of 0, leading to a low-variance result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports that experiments and throughput measurements use a single H200 GPU and provides measured inference throughput.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The research uses public benchmark datasets and existing open models for language-model acceleration experiments, does not involve human subjects or private data collection.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification:The paper proposes a training framework that makes diffusion LLM more efficient and does not directly imply positive/negative societal impacts.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not introduce a new scraped dataset or a new high-risk pretrained foundation model. It studies acceleration checkpoints for existing open language models.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the original papers for the main existing models, baselines, datasets, and evaluation tools used.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: Code and checkpoints will be released upon acceptance.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve crowdsourcing experiments or research with human subjects, so IRB or equivalent approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Ideas in this paper is original from human.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.